Lab 29 — Application Security: Validation, SAST and Code Signing

> **SY0-701 Objective 4.1** — Given a scenario, apply common security techniques to computing resources.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~15 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Find injection flaws in a Python app with static analysis (Bandit), fix them with input validation and parameterised queries, set secure cookie flags, sign the release with GPG, detect a tampered copy, and run untrusted code in a sandbox.

You'll build: A vulnerable app and its fixed version, before/after Bandit reports, a cookie server that issues `Secure; HttpOnly; SameSite` cookies, and a GPG-signed release tarball that fails verification when it is tampered with.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 10 (SQL injection) is helpful background; tools installed in step 1.

Background

- **Input validation** treats every input as untrusted. Prefer **allow-listing** (accept only what is known good) over deny-listing, always validate on the server side, and pair validation with **parameterised queries** so data can never become code.
- **Static code analysis (SAST)** reads source code without running it, so it finds flaws early in the SDLC. Bandit is a free SAST tool for Python. **Dynamic analysis (DAST)** tests the running app, and **fuzzing** throws random input at it.
- **Secure cookies** carry flags: `Secure` (HTTPS only), `HttpOnly` (no JavaScript access, which blunts XSS session theft) and `SameSite` (limits cross-site sending, which blunts CSRF).
- **Code signing** uses the publisher's private key to sign a hash of the code. Anyone with the public key can prove **integrity** (unchanged) and **authenticity** (who published it). It does not prove the code is free of bugs.
- **Sandboxing** runs untrusted code in an isolated environment with no network and a read-only system, so that malicious behaviour is contained.

Steps

1. Install the tools

Bandit is installed into its own Python virtual environment, which avoids the "externally managed environment" block that newer Ubuntu releases place on system-wide pip installs.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y python3-venv gnupg curl jq
```

```
python3 -m venv /opt/sast && /opt/sast/bin/pip install -q bandit
/opt/sast/bin/bandit --version | head -1
```

2. Create the vulnerable application

This customer-lookup tool contains five classic flaws: a hard-coded secret, SQL built by string formatting, a shell command built from input, a weak hash and `eval`. All the data is synthetic.

```
mkdir -p /opt/appsec/vuln
cat > /opt/appsec/vuln/lookup.py <<'EOF'
import hashlib, sqlite3, subprocess, sys

DB_PASSWORD = "Sup3rS3cret!"   # flaw 1: hard-coded secret (synthetic)

def init_db():
    con = sqlite3.connect(":memory:")
    con.execute("CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT, card TEXT)")
    con.executemany("INSERT INTO customers (name, card) VALUES (?, ?)",
                    [("alice", "4111-XXXX-XXXX-1111"), ("bob", "5500-XXXX-XXXX-0004"),
("carol", "3400-XXXX-XXXX-0009")])
    return con

def find_customer(con, name):
    query = "SELECT id, name, card FROM customers WHERE name = '%s'" % name   # flaw 2:
SQL injection
    return con.execute(query).fetchall()

def ping(host):
    return subprocess.check_output("ping -c 1 " + host, shell=True)           # flaw 3:
command injection

def token(user):
    return hashlib.md5(user.encode()).hexdigest()                             # flaw 4:
weak hash

def calc(expr):
    return eval(expr)                                                         # flaw 5:
code injection

if __name__ == "__main__":
    print(find_customer(init_db(), sys.argv[1]))
EOF
python3 /opt/appsec/vuln/lookup.py alice
```

```
[(1, 'alice', '4111-XXXX-XXXX-1111')]
```

3. Run static analysis with Bandit

Bandit parses the code into an abstract syntax tree and flags risky patterns. It exits non-zero when it finds issues, which is how a CI pipeline would block the merge.

```
cd /opt/appsec
/opt/sast/bin/bandit -r vuln -f json -o bandit-before.json || echo "Bandit found issues
(non-zero exit)"
```

```
jq -r '.results[] | "\(.issue_severity)\t\(.test_id)\tline \(.line_number)\t\
(.issue_text)"' bandit-before.json
jq '.metrics._totals | {high: ."SEVERITY.HIGH", medium: ."SEVERITY.MEDIUM", low:
."SEVERITY.LOW"}' bandit-before.json
```

Look for B602 (subprocess with `shell=True`), B608 (SQL built from strings), B307 (`eval`), B324 (MD5) and B105 (hard-coded password).

4. Confirm the SQL injection dynamically

A finding is only a hypothesis until you confirm it. Supply the classic tautology payload and watch every customer record come back. This is dynamic testing of the same flaw.

```
cd /opt/appsec
python3 vuln/lookup.py "x' OR '1'='1"
```

```
[(1, 'alice', '4111-XXXX-XXXX-1111'), (2, 'bob', '5500-XXXX-XXXX-0004'), (3, 'carol',
'3400-XXXX-XXXX-0009')]
```

5. Fix the code with validation and parameterisation

The fix applies an allow-list regular expression to every input, a bound parameter (`?`) for SQL, an argument list instead of a shell, SHA-256 instead of MD5, a secret read from the environment and `ast.literal_eval` instead of `eval`. Then re-scan with a gate that fails only on medium or high findings (`-ll`).

```
mkdir -p /opt/appsec/fixed
cat > /opt/appsec/fixed/lookup.py <<'EOF'
import ast, hashlib, os, re, sqlite3, subprocess, sys

DB_PASSWORD = os.environ.get("APP_DB_PASSWORD", "")   # secret injected at run time, never
in code
NAME_RE = re.compile(r"[a-z]{1,32}")                  # allow-list: lower-case letters
only
HOST_RE = re.compile(r"[A-Za-z0-9.-]{1,253}")

def init_db():
    con = sqlite3.connect(":memory:")
    con.execute("CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT, card TEXT)")
    con.executemany("INSERT INTO customers (name, card) VALUES (?, ?)",
                    [("alice", "4111-XXXX-XXXX-1111"), ("bob", "5500-XXXX-XXXX-0004"),
("carol", "3400-XXXX-XXXX-0009")])
    return con

def find_customer(con, name):
    if not NAME_RE.fullmatch(name):
        raise ValueError("invalid customer name")
    return con.execute("SELECT id, name, card FROM customers WHERE name = ?",
(name,)).fetchall()

def ping(host):
    if not HOST_RE.fullmatch(host):
        raise ValueError("invalid host")
    return subprocess.run(["/usr/bin/ping", "-c", "1", host], capture_output=True,
check=False).stdout
```

```
def token(user):
    return hashlib.sha256(user.encode()).hexdigest()

def calc(expr):
    return ast.literal_eval(expr)

if __name__ == "__main__":
    try:
        print(find_customer(init_db(), sys.argv[1]))
    except ValueError as err:
        print("REJECTED:", err)
EOF
cd /opt/appsec
python3 fixed/lookup.py alice
python3 fixed/lookup.py "x' OR '1'='1"
/opt/sast/bin/bandit -r fixed -ll && echo "SAST GATE PASSED: no medium or high findings"
```

```
[(1, 'alice', '4111-XXXX-XXXX-1111')]
REJECTED: invalid customer name
```

6. Issue cookies with secure flags

Start a tiny web server that sets one insecure cookie and one hardened cookie, then inspect the `Set-Cookie` headers.

```
cat > /opt/appsec/cookie_server.py <<'EOF'
from http.server import BaseHTTPRequestHandler, HTTPServer

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        if self.path == "/insecure":
            self.send_header("Set-Cookie", "session=lab-0001; Path=/")
        else:
            self.send_header("Set-Cookie", "session=lab-0001; Path=/; Secure; HttpOnly;
SameSite=Strict; Max-Age=900")
        self.end_headers()
        self.wfile.write(b"ok\n")

    def log_message(self, *args):
        pass

HTTPServer(("127.0.0.1", 8088), Handler).serve_forever()
EOF
python3 /opt/appsec/cookie_server.py & SRV=$!
sleep 1
curl -s -D - -o /dev/null http://127.0.0.1:8088/insecure | grep -i '^set-cookie'
curl -s -D - -o /dev/null http://127.0.0.1:8088/login | grep -i '^set-cookie'
kill "$SRV"
```

```
Set-Cookie: session=lab-0001; Path=/
Set-Cookie: session=lab-0001; Path=/; Secure; HttpOnly; SameSite=Strict; Max-Age=900
```

7. Sign the release with GPG

Create a lab-only signing key (no passphrase, for the lab only), package the fixed app, publish a SHA-256 checksum, and produce a detached ASCII-armoured signature.

```
export GNUPGHOME=/opt/appsec/gnupg
mkdir -p -m 700 "$GNUPGHOME"
cat > /opt/appsec/keyparams <<'EOF'
%no-protection
Key-Type: RSA
Key-Length: 3072
Key-Usage: sign
Name-Real: C628 Release Signing (LAB ONLY)
Name-Email: release@lab.example
Expire-Date: 1y
%commit
EOF
gpg --batch --gen-key /opt/appsec/keyparams
cd /opt/appsec
tar -czf lookup-1.0.tar.gz -C fixed lookup.py
sha256sum lookup-1.0.tar.gz > lookup-1.0.tar.gz.sha256
gpg --batch --yes --armor --detach-sign lookup-1.0.tar.gz
gpg --verify lookup-1.0.tar.gz.asc lookup-1.0.tar.gz
```

```
gpg: Good signature from "C628 Release Signing (LAB ONLY) <release@lab.example>"
```

8. Detect a tampered release

An attacker who modifies the package cannot produce a valid signature without the private key, so verification fails.

```
export GNUPGHOME=/opt/appsec/gnupg
cd /opt/appsec
cp lookup-1.0.tar.gz lookup-1.0-tampered.tar.gz && echo "backdoor" >> lookup-1.0-
tampered.tar.gz
gpg --verify lookup-1.0.tar.gz.asc lookup-1.0-tampered.tar.gz || echo "SIGNATURE CHECK
FAILED - do not install"
sha256sum -c lookup-1.0.tar.gz.sha256
```

```
gpg: BAD signature from "C628 Release Signing (LAB ONLY) <release@lab.example>"
SIGNATURE CHECK FAILED - do not install
lookup-1.0.tar.gz: OK
```

9. Run untrusted code in a sandbox

Use systemd's sandboxing to run a script with no network, a read-only operating system and no privilege escalation. The script tries to write to `/etc` and to call out to the internet, and both attempts fail.

```
systemd-run --quiet --wait --pipe \
  -p PrivateNetwork=yes -p ProtectSystem=strict -p ProtectHome=yes -p PrivateTmp=yes -p
NoNewPrivileges=yes \
  /bin/sh -c 'touch /etc/pwned 2>&1; curl -sS -m 3 http://example.com 2>&1 | head -1; echo
```

```
"sandboxed run finished"'
test -e /etc/pwned && echo "ESCAPED" || echo "Host /etc untouched — sandbox held"
```

```
touch: cannot touch '/etc/pwned': Read-only file system
curl: (6) Could not resolve host: example.com
sandboxed run finished
Host /etc untouched — sandbox held
```

Test it

```
export GNUPGHOME=/opt/appsec/gnupg; cd /opt/appsec
/opt/sast/bin/bandit -q -r fixed -ll; echo "bandit-fixed exit=$?"
python3 fixed/lookup.py "x' OR '1'='1"
gpg --verify lookup-1.0.tar.gz.asc lookup-1.0.tar.gz 2>&1 | grep -o 'Good signature'
gpg --verify lookup-1.0.tar.gz.asc lookup-1.0-tampered.tar.gz 2>&1 | grep -o 'BAD
signature'
```

Expected: `bandit-fixed exit=0`, `REJECTED: invalid customer name`, `Good signature` for the original and `BAD signature` for the tampered copy.

Troubleshooting

Symptom	Fix
`error: externally-managed-environment` from pip	Use the venv path `/opt/sast/bin/pip` exactly as shown. Do not use the system `pip3`.
`gpg: signing failed: No secret key`	`GNUPGHOME` is not set in this shell. Run `export GNUPGHOME=/opt/appsec/gnupg` again.
`Address already in use` on port 8088	A previous cookie server is still running. Run `pkill -f cookie_server.py` and repeat step 6.

Challenge

Add a CI-style gate script that runs Bandit with `-ll`, verifies the GPG signature and the SHA-256 checksum, and only then extracts the tarball to `/opt/release/`. Prove that it refuses the tampered tarball.

Reflection

Your developers sign every release, yet a customer was still compromised by a signed build. What does code signing prove, what does it not prove, and which other 4.1 application-security technique would have caught the flaw earlier?

Exam tips

- Static analysis = source code, not running (SAST). Dynamic analysis = running app (DAST, fuzzing). The exam often asks which one finds flaws earliest: static.
- Code signing proves integrity and publisher authenticity, not safety. A stolen signing key signs malware just as well.

- `HttpOnly` stops JavaScript reading the cookie (XSS); `Secure` forces HTTPS; `SameSite` limits cross-site requests (CSRF).
- Server-side allow-list validation plus parameterised queries is the textbook fix for injection; client-side validation alone can be bypassed.

References

- OWASP Input Validation Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- OWASP Session Management Cheat Sheet (cookie attributes) — https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- Bandit documentation — https://bandit.readthedocs.io
- GnuPG manual, unattended key generation — https://www.gnupg.org/documentation/manuals/gnupg/Unattended-GPG-key-generation.html
- systemd.exec sandboxing options — https://www.freedesktop.org/software/systemd/man/latest/systemd.exec.html

---